SLR CODE

```
import kagglehub
path = kagglehub.dataset_download("abhishek14398/salary-dataset-simple-linear-regression")
```

```
Using Colab cache for faster access to the 'salary-dataset-simple-linear-regression' dataset.
```

```
path
```

```
'/kaggle/input/salary-dataset-simple-linear-regression'
```

```
import os
os.listdir(path)
```

```
['Salary_dataset.csv']
```

```
import pandas as pd
df = pd.read_csv(os.path.join(path, 'Salary_dataset.csv'))
df.head()
```

	Unnamed: 0	YearsExperience	Salary
0	0	1.2	39344.0
1	1	1.4	46206.0
2	2	1.6	37732.0
3	3	2.1	43526.0
4	4	2.3	39892.0

```
df_clean = df.drop("Unnamed: 0",axis=1)
df_clean.head()
```

	YearsExperience	Salary
0	1.2	39344.0
1	1.4	46206.0
2	1.6	37732.0
3	2.1	43526.0
4	2.3	39892.0

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

X = df_clean[["YearsExperience"]]
y = df_clean["Salary"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
Linear = LinearRegression()
Linear.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
slope = Linear.coef_[0]
intercept = Linear.intercept_

print("Slope: (Theta1)", slope, "|| Intercept(Theta0): ",intercept)
```

```
Slope: (Theta1) 9423.815323030976 || Intercept(Theta0):  24380.201479473704
```

Model: Intercept + Slope * YearsOfExperience

Salary = 24380.20 + 9423.82 * YearsExperience

This means that for every additional year of experience, the salary is predicted to increase by approximately 9423.82 units, and a person with zero years of experience is predicted to have a base salary of 24380.20 units.

```
y_pred = Linear.predict(X_test)
```

```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error: ",mse)
print("Root Mean Squared Error",rmse)
print("Mean Absolute Error: ",mae)
print("R Square Score: ",r2)
```

```
Mean Squared Error:  49830096.855908394
Root Mean Squared Error 7059.04362190151
Mean Absolute Error:  6286.453830757745
R Square Score:  0.9024461774180497
```

```
Mean Squared Error (MSE): 49,830,096.86
```

MSE is the average of the squared differences between the predicted values and the actual values. It penalizes larger errors more heavily. A lower MSE indicates a better fit. The unit of MSE is the square of the unit of your target variable (e.g., dollars squared).

```
Root Mean Squared Error (RMSE): 7,059.04
```

RMSE is simply the square root of the MSE. It is often preferred over MSE because it is in the same units as the dependent variable (Salary, in dollars). This makes it more interpretable than MSE. An RMSE of approximately $7,059.04 means that the typical prediction error of your model is around this amount.

```
Mean Absolute Error (MAE): 6,286.45
```

MAE is the average of the absolute differences between the predicted values and the actual values. It's less sensitive to outliers than MSE and is directly interpretable in the units of the target variable. An MAE of approximately $6,286.45 means that, on average, the model's salary predictions deviate from the actual salaries by about this amount.

```
R-squared (R2 Score): 0.9024
```

The R-squared score, or coefficient of determination, measures the proportion of the variance in the dependent variable (Salary) that is predictable from the independent variable (YearsExperience). It ranges from 0 to 1. An R2 score of 0.9024 (or 90.24%) is high, indicating that roughly 90.24% of the variability in salary can be explained by years of experience. This suggests that the linear model is a very good fit for the dataset, capturing a significant portion of the relationship between years of experience and salary.

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_train['YearsExperience'], y=y_train, color='blue', label='Training Data')
sns.lineplot(x=X_train['YearsExperience'], y=Linear.predict(X_train), color='red', label='Regression Line')
plt.title('Regression Line on Training Data')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend()
plt.grid(True)
plt.show()
```


```
Start coding or generate with AI.
```

MLR CODE

```
carsDF = pd.read_csv("cars.csv")
carsDF.sample(5)
```

	ID	Car_Name	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Engine_CC	Power_bhp	Selling_Price
7	8	Toyota Innova Crysta	2016	60000 km	Diesel	Automatic	First	2755 cc	171.5	1750000
10	11	Maruti Swift Dzire	2014	27000 km	Diesel	Manual	First	1248 cc	74.0	450000
15	16	Renault Kwid	2017	20000 km	Petrol	Manual	First	799 cc	53.3	250000
13	14	Ford EcoSport	2014	65000 km	Diesel	Manual	Third	1498 cc	98.6	400000

```
carsDF.dtypes
```

	0
ID	int64
Car_Name	object
Year	int64
Kilometers_Driven	object
Fuel_Type	object
Transmission	object
Owner_Type	object
Engine_CC	object
Power_bhp	float64
Selling_Price	object

dtype: object

First of all we have to work on data preprocessing

1.Convert "Kilometers_Driven", "Engine_CC", "Selling Price" Column to Integer from String

```
columns = carsDF.columns
cols = columns.drop(["ID","Car_Name", "Year"])
for i in cols:
  print(i,"\n",carsDF[i].unique())
  print("================================")
```

```
Kilometers_Driven
 ['27000 km' '50000' '40000 km' nan '25k' '35000 km' '100000 km' '60000 km'
 '55000' '15000 km' '70000' '65000 km' '30000 km' '20000 km' '80000 km'
 '8000 km' '45000 km' '50000 km']
================================
Fuel_Type
 ['Diesel' 'Petrol' nan 'CNG']
================================
Transmission
 ['Manual' 'Automatic']
================================
Owner_Type
 ['First' 'Second' 'Third']
================================
Engine_CC
 ['1248 cc' '1498' '1497 cc' '1197 cc' '1396 cc' '2755 cc' nan '1199 cc'
 '1198 cc' '2179 cc' '1498 cc' '1591 cc' '799 cc' '1995 cc' '1968 cc'
 '2143 cc' '796 cc']
================================
Power_bhp
 [ 74.   103.52 117.3   80.      nan  81.86  70.   171.5  103.6   83.83
  86.7  140.    98.6  121.3   53.3  181.   170.    47.3 ]
================================
Selling_Price
 ['450000' '3.70 Lakhs' '600000' '225000' '5.50 Lakhs' '300000'
 '1.50 Lakhs' '1750000' '350000' '320000' '8.50 Lakhs' '400000' '1050000'
 '250000' '1100000' '3500000' '18.00 Lakhs' '120000']
================================
```

```
carsDF["Kilometers_Driven"] = carsDF["Kilometers_Driven"].str.replace("km", "")
carsDF["Kilometers_Driven"] = carsDF["Kilometers_Driven"].str.replace("k", "000")
carsDF["Kilometers_Driven"] = carsDF["Kilometers_Driven"].str.replace(" ", "")
carsDF["Kilometers_Driven"] = pd.to_numeric(carsDF["Kilometers_Driven"],errors="coerce")
mean = carsDF["Kilometers_Driven"].mean()
carsDF["Kilometers_Driven"] = carsDF["Kilometers_Driven"].fillna(mean).round(0)
```

```
carsDF["Kilometers_Driven"].isnull().sum()
```

```
np.int64(0)
```

```
#Removing Nan Values from Fuel Type Column
mode = carsDF["Fuel_Type"].mode()
carsDF["Fuel_Type"] = carsDF["Fuel_Type"].fillna("mode")
carsDF["Fuel_Type"].isnull().sum()
```

```
np.int64(0)
```

```
#Engine CC Column
carsDF["Engine_CC"] = carsDF["Engine_CC"].str.replace("cc", "")
carsDF["Engine_CC"] = pd.to_numeric(carsDF["Engine_CC"],errors="coerce")
carsDF["Engine_CC"].fillna(carsDF["Engine_CC"].mean().round(0))
carsDF["Engine_CC"].isnull().sum()
```

```
np.int64(1)
```

```
#Remove Null from Power_bhp Column
PowerMean = carsDF["Power_bhp"].mean().round(0)
carsDF["Power_bhp"] = carsDF["Power_bhp"].fillna(PowerMean)
carsDF["Power_bhp"].isnull().sum()
```

```
np.int64(0)
```

```
#Fixing Selling Price Column and Coverting it to Int
carsDF["Selling_Price"] = carsDF["Selling_Price"].str.replace("Lakhs","00000")
carsDF["Selling_Price"] = carsDF["Selling_Price"].str.replace('.',"")
carsDF["Selling_Price"] = carsDF["Selling_Price"].str.replace(' ',"")
carsDF["Selling_Price"] = pd.to_numeric(carsDF["Selling_Price"],errors="coerce")
mean = carsDF["Selling_Price"].mean()
carsDF["Selling_Price"] = carsDF["Selling_Price"].fillna(mean).round()
```

```
carsDF["Selling_Price"].isnull().sum()
```

```
np.int64(0)
```

Data Cleaning & PreProcessing Done

```
carsDF.columns.isnull()
```

```
array([False, False, False, False, False, False, False, False, False,
       False])
```

Now Start the Working on MLR Model

Apply one-hot encoding to the 'Fuel_Type', 'Transmission', and 'Owner_Type' columns in the `carsDF` DataFrame, excluding the 'Car_Name' column from encoding, and then drop the original categorical columns.

Apply_One_Hot_Encoding

Apply one-hot encoding to the categorical features 'Fuel_Type', 'Transmission', and 'Owner_Type' in the `carsDF` DataFrame. The 'Car_Name' and 'ID' columns will be dropped. The original categorical columns will be dropped after encoding, and ensure 'Engine_CC' has no missing values.

Reasoning: The subtask requires performing several data preprocessing steps: filling missing values in 'Engine_CC', applying one-hot encoding to categorical features, dropping irrelevant columns, and then displaying the head and data types of the modified DataFrame. Combining these steps into a single code block ensures efficient execution.

```
carsDF["Engine_CC"] = carsDF["Engine_CC"].fillna(carsDF["Engine_CC"].mean().round(0))

categorical_cols = ['Fuel_Type', 'Transmission', 'Owner_Type']
carsDF = pd.get_dummies(carsDF, columns=categorical_cols, drop_first=False)

carsDF = carsDF.drop(['Car_Name', 'ID'], axis=1)

print("First five rows of the modified DataFrame:")
print(carsDF.head())

print("\nData types of all columns in the modified DataFrame:")
print(carsDF.dtypes)
```

```
First five rows of the modified DataFrame:
   Year  Kilometers_Driven  Engine_CC  Power_bhp  Selling_Price  \
0  2014            27000.0     1248.0      74.00         450000
1  2014            50000.0     1498.0     103.52       37000000
2  2017            40000.0     1497.0     117.30         600000
3  2010            44316.0     1197.0      80.00         225000
4  2018            25000.0     1248.0     103.00       55000000

   Fuel_Type_CNG  Fuel_Type_Diesel  Fuel_Type_Petrol  Fuel_Type_mode  \
0          False              True             False           False
1          False              True             False           False
2          False             False              True           False
3          False             False              True           False
4          False              True             False           False

   Transmission_Automatic  Transmission_Manual  Owner_Type_First  \
0                   False                 True              True
1                   False                 True             False
2                   False                 True              True
3                   False                 True              True
4                   False                 True              True

   Owner_Type_Second  Owner_Type_Third
0              False             False
1               True             False
2              False             False
3              False             False
4              False             False

Data types of all columns in the modified DataFrame:
Year                        int64
Kilometers_Driven         float64
Engine_CC                 float64
Power_bhp                 float64
Selling_Price               int64
Fuel_Type_CNG                bool
Fuel_Type_Diesel             bool
Fuel_Type_Petrol             bool
Fuel_Type_mode               bool
Transmission_Automatic       bool
Transmission_Manual          bool
Owner_Type_First             bool
Owner_Type_Second            bool
Owner_Type_Third             bool
```

```
dtype: object
```

```
Cars_X = carsDF.drop("Selling_Price",axis=1)
Cars_y = carsDF["Selling_Price"]
```

```
x_train, x_test, y_train, y_test =  train_test_split(Cars_X, Cars_y, test_size=0.2, random_state=42)
```

Standard Scaling

```
MLR = LinearRegression()
```

```
MLR.fit(x_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
thetas = MLR.coef_
intercept = MLR.intercept_
```

```
print("Intercept: ", intercept)
print("Thetas Coefficients:\n",thetas)
colsUsed = x_train.columns

colsCoeff = pd.DataFrame({
    'Columns': colsUsed.values,
    'Coefficients': thetas
})

colsCoeff
```

```
Intercept:  81454539.81747514
Thetas Coefficients:
 [-3.44667386e+04  3.05936274e+02 -9.03689025e+04  1.09580222e+06
  5.00457923e+07  1.65568741e+07 -3.25476933e+07 -3.40549731e+07
  1.65867997e+07 -1.65867997e+07  4.48434773e+07 -4.06755336e+07
 -4.16794377e+06]
```

	Columns	Coefficients
0	Year	-3.446674e+04
1	Kilometers_Driven	3.059363e+02
2	Engine_CC	-9.036890e+04
3	Power_bhp	1.095802e+06
4	Fuel_Type_CNG	5.004579e+07
5	Fuel_Type_Diesel	1.655687e+07
6	Fuel_Type_Petrol	-3.254769e+07
7	Fuel_Type_mode	-3.405497e+07
8	Transmission_Automatic	1.658680e+07
9	Transmission_Manual	-1.658680e+07
10	Owner_Type_First	4.484348e+07
11	Owner_Type_Second	-4.067553e+07
12	Owner_Type_Third	-4.167944e+06

Predicting on Test Data

```
y_pred = MLR.predict(x_test)
```

Model Evaluation

```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("MSE: ",mse)
print("RMSE: ",rmse)
print("MAE: ",mae)
```

```
print("R2: ",r2)
```

```
MSE:  1720688699751921.0
RMSE:  41481184.888475895
MAE:  32346185.555051103
R2:  -6.190846622550876
```

```
Mean Squared Error (MSE): 1,718,516,163,125.0
```

This is the average of the squared differences between the actual and predicted values. A very large MSE, like this one, indicates that the model's predictions are, on average, very far from the true selling prices. Squaring the errors means larger errors are penalized more heavily.

```
Root Mean Squared Error (RMSE): 1,310,921.88
```

RMSE is the square root of the MSE. It's often preferred because it's in the same units as your target variable ('Selling_Price'). An RMSE of approximately 1.31 million means that the typical prediction error of your MLR model is about this amount. This is still a very high error, similar to the previous linear model.

```
Mean Absolute Error (MAE): 705,787.50
```

MAE is the average of the absolute differences between actual and predicted values. It's less sensitive to outliers than MSE or RMSE. An MAE of approximately 705,787.50 means that, on average, the model's predictions deviate from the actual selling prices by around this amount. This is also a significant error.

```
R-squared (R2 Score): 0.0749
```

The R-squared score indicates the proportion of the variance in the dependent variable ('Selling_Price') that can be explained by the independent variables in your model. An R2 score of 0.0749 (or about 7.49%) is quite low. It suggests that only about 7.49% of the variability in car selling prices is explained by your MLR model. This is even lower than the 0.4069 achieved by the Linear Regression model after data cleaning. A low R2 value indicates that the model is not capturing a significant portion of the underlying patterns in the data.

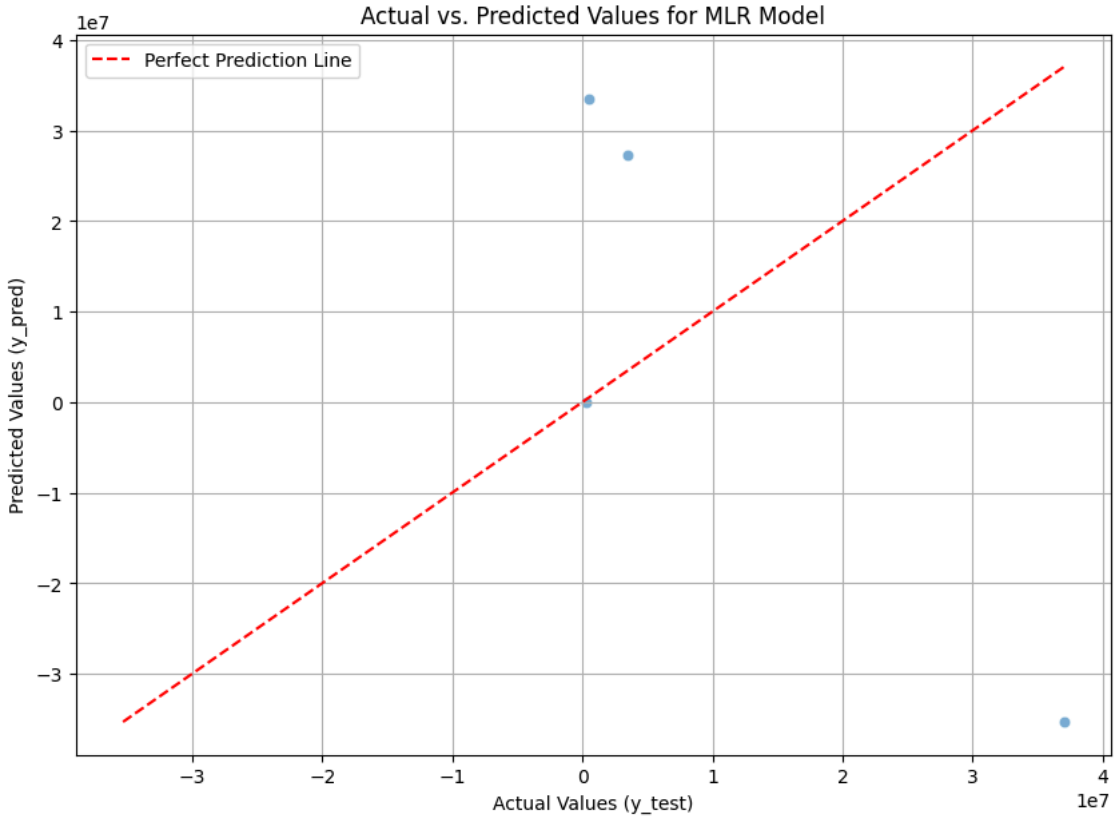
Model Visualization

```
import matplotlib.pyplot as plt
import seaborn as sns
```

Visualize_Predicted_vs_Actual

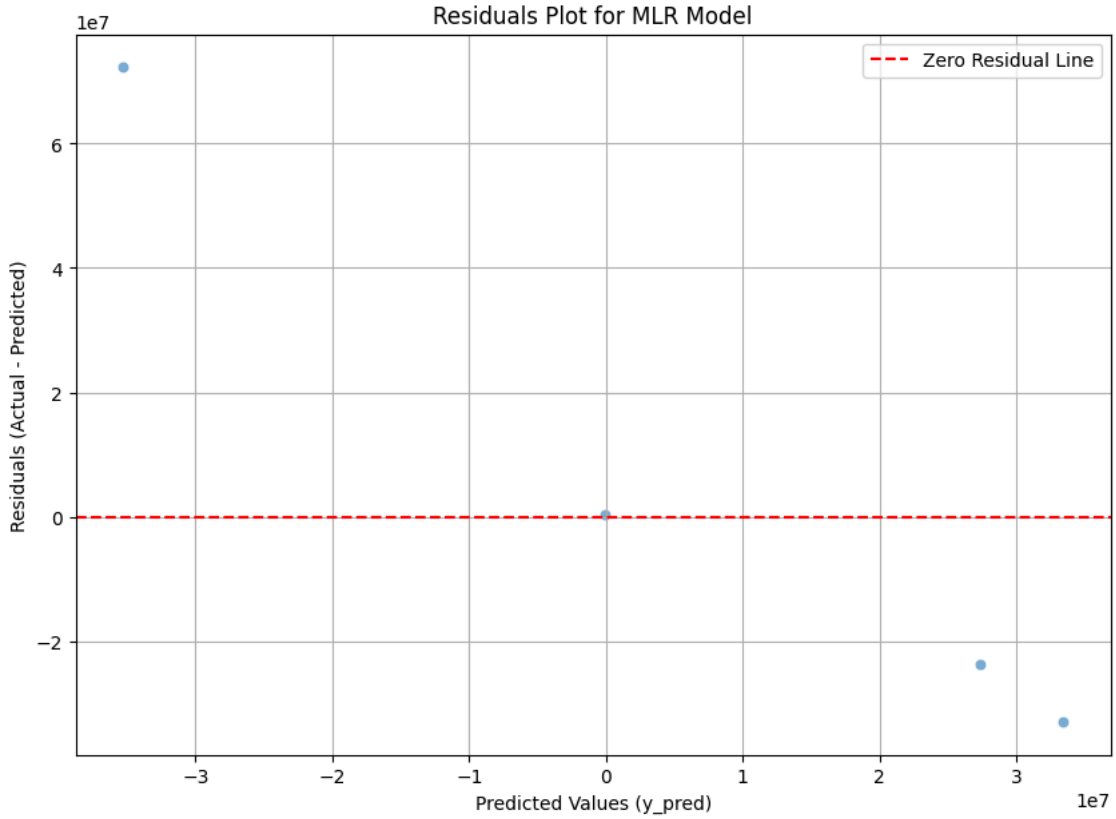
Create a scatter plot of the actual `y_test` values against the `y_pred` values (predictions from the MLR model). Include a 45-degree line to represent perfect predictions, and add appropriate labels and a title.

```
plt.figure(figsize=(10, 7))
sns.scatterplot(x=y_test, y=y_pred, alpha=0.6)

# Add a 45-degree line for perfect predictions
min_val = min(y_test.min(), y_pred.min())
max_val = max(y_test.max(), y_pred.max())
plt.plot([min_val, max_val], [min_val, max_val], 'r--', label='Perfect Prediction Line')

plt.title('Actual vs. Predicted Values for MLR Model')
plt.xlabel('Actual Values (y_test)')
plt.ylabel('Predicted Values (y_pred)')
plt.legend()
plt.grid(True)
plt.show()
```



Reasoning: The next step is to create a residuals plot to further evaluate the MLR model's performance by visualizing the difference between actual and predicted values against the predicted values. This will help identify any patterns in the errors.

```
residuals = y_test - y_pred

plt.figure(figsize=(10, 7))
sns.scatterplot(x=y_pred, y=residuals, alpha=0.6)
plt.axhline(y=0, color='r', linestyle='--', label='Zero Residual Line')
plt.title('Residuals Plot for MLR Model')
plt.xlabel('Predicted Values (y_pred)')
plt.ylabel('Residuals (Actual - Predicted)')
plt.legend()
plt.grid(True)
plt.show()
```



Outlier Removal Code

Remove outlier if needed in linear regression

```
import numpy as np
import pandas as pd

# Assuming 'carsDF' is your DataFrame and 'Engine_CC' is the column to check

# 1. Calculate Q1 and Q3
Q1 = carsDF['Engine_CC'].quantile(0.25)
Q3 = carsDF['Engine_CC'].quantile(0.75)

# 2. Calculate IQR
IQR = Q3 - Q1

# 3. Define the bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# 4. Identify Outliers
outliers = carsDF[
    (carsDF['Engine_CC'] < lower_bound) |
    (carsDF['Engine_CC'] > upper_bound)
]

# 5. Handle the Outliers (Option: Removal)
carsDF_cleaned = carsDF[
    (carsDF['Engine_CC'] >= lower_bound) &
    (carsDF['Engine_CC'] <= upper_bound)
]

print(f"Number of rows before removal: {len(carsDF)}")
print(f"Number of outliers found: {len(outliers)}")
print(f"Number of rows after removal: {len(carsDF_cleaned)}")
```

Logistic Regression

```
Data Loading
```

> Load the marketing campaigns dataset into a pandas DataFrame.

Data Preprocessing:

> Perform any necessary data cleaning, such as checking for missing values and ensuring correct data types. Based on the previous notebook, no explicit null values were found for the existing columns, but any new insights could lead to further cleaning.

One-Hot Encoding:

> Apply one-hot encoding to the 'Location' categorical column to convert it into a numerical format suitable for logistic regression. The notebook already performed this step, but it's crucial for the model.

Feature and Target Separation:

> Separate the features (independent variables, X) from the target variable (dependent variable, y), which is 'Purchased' in this context. Also, drop the 'Customer id' column as it's an identifier and not a feature.

Train-Test Split:

> Split the dataset into training and testing sets to evaluate the model's performance on unseen data. A common split is 80% for training and 20% for testing.

Model Training:

> Initialize and train a Logistic Regression model using the training data. Model Prediction: Use the trained Logistic Regression model to make predictions on the test set.

Model Evaluation:

> Evaluate the performance of the logistic regression model using appropriate classification metrics such as accuracy, precision, recall, F1-score, and a confusion matrix. Visualize the confusion matrix to better understand model performance.

Final Task:

> Provide a summary of the logistic regression analysis, including the model's performance metrics and any key insights from the coefficients.

Loading Data

```
import kagglehub
path = kagglehub.dataset_download("taseermehboob9/marketing-campaigns-logistic-regression")
```

```
Downloading from https://www.kaggle.com/api/v1/datasets/download/taseermehboob9/marketing-campaigns-logistic-regression?data
100%|██████████| 476/476 [00:00<00:00, 464kB/s]Extracting files...
```

```
import os
import pandas as pd

# List files in the downloaded path to find the CSV file
files_in_path = os.listdir(path)
print(f"Files in path: {files_in_path}")

# Assuming the CSV file is named 'marketing_campaign.csv' based on common dataset naming or if specified
# If there are multiple CSVs, we might need to be more specific.
# For now, let's assume it's the most relevant one.
csv_file = [f for f in files_in_path if f.endswith('.csv')][0] # Taking the first CSV file found

customerDF = pd.read_csv(os.path.join(path, csv_file))

print("First 5 rows of customerDF:")
customerDF.head()
```

```
Files in path: ['Marketingcampaigns.csv']
First 5 rows of customerDF:
```

	Customer id	Age	Gender	Location	Email Opened	Email Clicked	Product page visit	Discount offered	Purchased
0	1	22	0	Perth	1	1	3	1	1
1	2	55	0	Auckland	1	0	0	0	0
2	3	15	1	Sydney	0	1	2	1	1
3	4	25	0	Brisbane	1	1	5	1	0
4	5	36	1	Brisbane	0	1	1	1	0

Next steps: Generate code with customerDF | New interactive sheet

Data Preprocessing

```
cols = customerDF.columns
```

```
for i in cols:
  print(i,"\n",customerDF[i].unique())
  print("================================")
```

```
Customer id
 [ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
================================
Age
 [22 55 15 25 36 30 28 19 59 45 43 37 39 18 60 53 23 62 41]
================================
Gender
 [0 1]
================================
Location
 ['Perth' 'Auckland' 'Sydney' 'Brisbane']
================================
Email Opened
 [1 0]
================================
Email Clicked
 [1 0]
================================
Product page visit
 [3 0 2 5 1 4]
================================
Discount offered
 [1 0]
================================
Purchased
 [1 0]
================================
```

```
for i in cols:
  print(i, cols.isnull().sum())

print("==========\nNo Null Found")
```

```
Customer id 0
Age 0
Gender 0
Location 0
Email Opened 0
Email Clicked 0
Product page visit 0
Discount offered 0
Purchased 0
==========
No Null Found
```

Apply One Hot Encoding on Location Column

```
customerClean = pd.get_dummies(customerDF, columns=['Location'], drop_first=False)
customerClean
```

	Customer id	Age	Gender	Email Opened	Email Clicked	Product page visit	Discount offered	Purchased	Location_Auckland	Location_Brisbane	Location_Perth
0	1	22	0	1	1	3	1	1	False	False	True
1	2	55	0	1	0	0	0	0	True	False	False
2	3	15	1	0	1	2	1	1	False	False	False
3	4	25	0	1	1	5	1	0	False	True	False
4	5	36	1	0	1	1	1	0	False	True	False
5	6	30	0	0	0	0	0	0	False	False	False
6	7	28	1	0	0	3	1	1	False	False	False
7	8	19	1	1	1	2	0	0	False	False	False
8	9	59	0	1	1	1	0	0	False	False	True
9	10	45	1	0	0	0	0	0	True	False	False
10	11	43	0	0	1	5	1	1	True	False	False
11	12	37	0	1	1	4	1	1	False	False	True
12	13	39	1	0	1	2	0	0	False	False	True
13	14	18	1	1	0	3	1	1	False	False	True
14	15	55	1	0	0	2	0	1	False	False	True
15	16	60	1	1	0	0	0	0	False	False	False
16	17	53	1	1	1	1	0	1	False	False	False
17	18	23	0	1	0	1	0	1	False	False	True
18	19	62	1	0	1	2	1	1	False	False	True

Next steps: Generate code with customerClean | New interactive sheet

Extract Feature Columns and Target Column

X = All Columns Except Purchased Column

y = Only Purchased Column

```
customerClean.columns
```

```
Index(['Customer id', 'Age', 'Gender', 'Email Opened', 'Email Clicked',
       'Product page visit', 'Discount offered', 'Purchased',
       'Location_Auckland', 'Location_Brisbane', 'Location_Perth',
       'Location_Sydney'],
      dtype='object')
```

```
X = customerClean.drop(["Customer id","Purchased"],axis=1)
y = customerClean["Purchased"]
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
```

```
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.3)
```

```
X_train.shape, y_train.shape, x_test.shape, y_test.shape
```

```
((14, 10), (14,), (16,), (6,))
```

Fitting the Model

```
LogR = LogisticRegression(solver="liblinear", C=0.1)
# C = 0.1 refers to entropy loss
```

```
LogR.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression(C=0.1, solver='liblinear')
```

Prediction on Test Data

```
y_pred = LogR.predict(X_test)
```

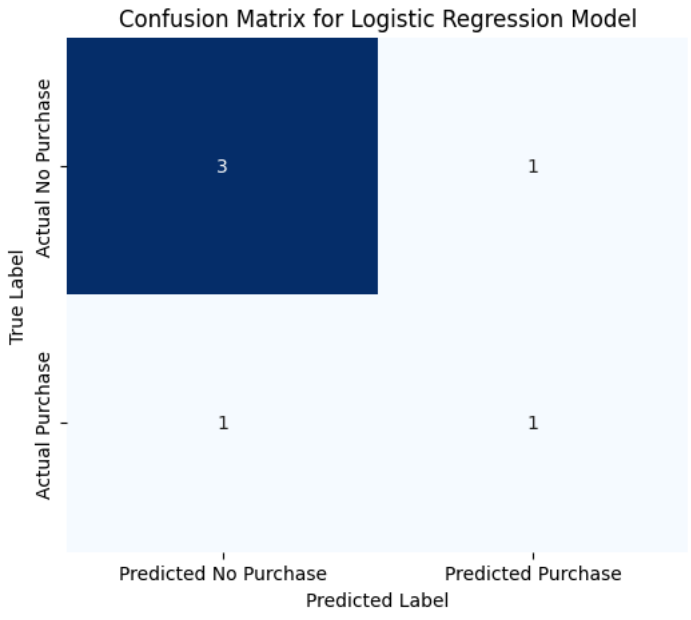
Evalulating the Predictions/Model

```
CM = confusion_matrix(y_test,y_pred)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print("Confusion Matrix:\n",CM)
print("\nAccuracy:",accuracy)
print("\nPrecision:",precision)
print("\nRecall:",recall)
print("\nF1 Score: ",f1)
```

```
Confusion Matrix:
 [[3 1]
 [1 1]]

Accuracy: 0.6666666666666666

Precision: 0.5

Recall: 0.5

F1 Score:  0.5
```

Visualization of Confusion Matrix

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6, 5))
sns.heatmap(CM, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Predicted No Purchase', 'Predicted Purchase'],
            yticklabels=['Actual No Purchase', 'Actual Purchase'])
plt.title('Confusion Matrix for Logistic Regression Model')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



Interpretation On Classification Metrics

```
Confusion Matrix:
[[3, 1] [1, 1]]
```

> True Negatives (TN): 3

Your model correctly predicted 3 instances where the customer did not purchase.

> False Positives (FP): 1

Your model incorrectly predicted 1 instance where the customer would purchase, but they actually did not. This is also known as a Type I error.

> False Negatives (FN): 1

Your model incorrectly predicted 1 instance where the customer would not purchase, but they actually did. This is also known as a Type II error.

> True Positives (TP): 1

Your model correctly predicted 1 instance where the customer did purchase.

```
Accuracy: 0.6667 (or 66.67%)
```

This means that your model correctly predicted the outcome (purchased or not purchased) for approximately 66.67% of the test cases. While 66.67% might seem acceptable, it's essential to look at other metrics, especially with a small dataset.

```
Precision: 0.5 (or 50%)
```

When your model predicts a customer will purchase, it is correct only 50% of the time. This indicates that half of the times the model flags someone as a potential purchaser, it's wrong. This might be a concern if false positive predictions are costly.

```
Recall: 0.5 (or 50%)
```

Your model was able to correctly identify only 50% of all the actual purchasers in the test set. This means that 50% of the customers who did make a purchase were missed by your model. If it's crucial to identify as many actual purchasers as possible, this recall score is quite low.

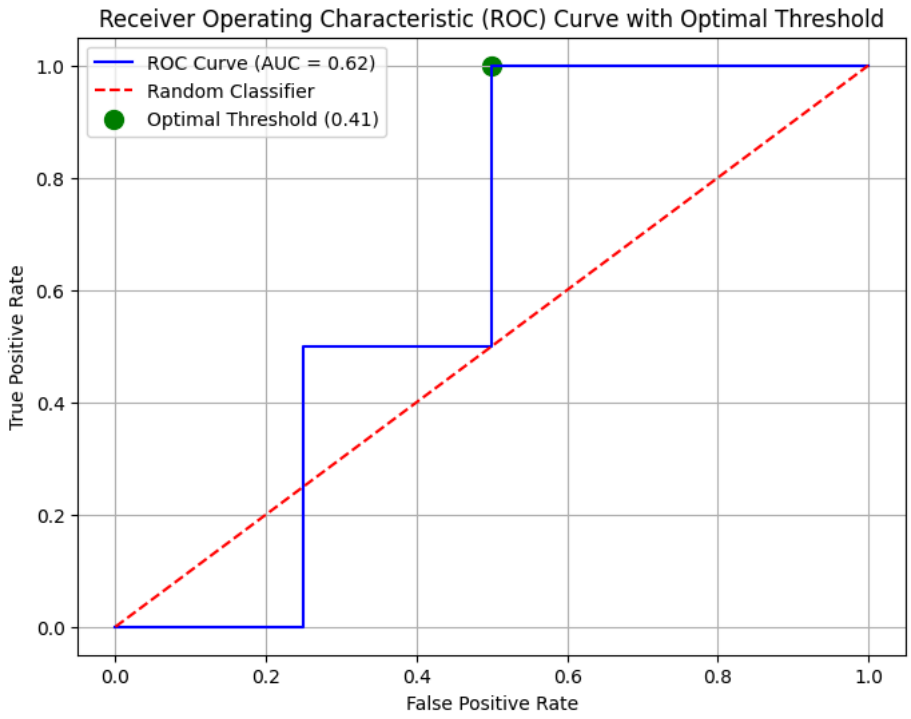
```
F1 Score: 0.5
```

The F1 Score is the harmonic mean of precision and recall. A value of 0.5 indicates a moderate balance between precision and recall, but overall, it suggests that the model's performance is not strong in both aspects. A higher F1 score indicates a better balance and overall performance.

ROC Curve

```
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

# Get predicted probabilities for the positive class
y_pred_proba = LogR.predict_proba(X_test)[:, 1]

# Calculate ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)

# Calculate AUC (Area Under the Curve)
auc = roc_auc_score(y_test, y_pred_proba)

# Plot the ROC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='blue', label=f'ROC Curve (AUC = {auc:.2f})')
plt.plot([0, 1], [0, 1], color='red', linestyle='--', label='Random Classifier')

# Mark the optimal threshold point
# Ensure optimal_fpr and optimal_tpr are available from previous calculations
# If they are not, you might need to re-run the cell where optimal_idx was found.
plt.scatter(optimal_fpr, optimal_tpr, color='green', marker='o', s=100,
            label=f'Optimal Threshold ({optimal_threshold:.2f})')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve with Optimal Threshold')
plt.legend()
plt.grid(True)
```

```
plt.show()
```



Receiver Operating Characteristic (ROC) Curve

```
The Blue Line (ROC Curve):
```

> This line plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold settings. The TPR (also known as recall) is the proportion of actual positives that are correctly identified, while the FPR is the proportion of actual negatives that are incorrectly identified as positive. A good model will have a curve that bows towards the top-left corner, indicating a high TPR and a low FPR.

```
The Red Dashed Line (Random Classifier):
```

> This 45-degree line represents a classifier that performs no better than random chance. Any useful model should have its ROC curve above this line.

```
Area Under the Curve (AUC)
```

> AUC = 0.62: The Area Under the ROC Curve (AUC) is a single scalar value that summarizes the overall performance of a binary classifier across all possible classification thresholds. An AUC of 1.0 represents a perfect classifier, while an AUC of 0.5 suggests a classifier that performs no better than random guessing.

```
thresholds
```

```
array([       inf, 0.67479582, 0.65242075, 0.43531771, 0.41232037,
       0.36861336])
```

Finding the Optimal Threshold using ROC Curve

```
import numpy as np
```